

데이터관리와 분석

Project 2 보고서

DB Mining · Document Search Engine · Classification

팀 13조

2026년 6월

목 차

1. 서 론.....	4
1.1 프로젝트 개요	4
1.2 보고서 구성	4
2. PART I — 연관 규칙 분석.....	5
2.1 분석 목적.....	5
2.2 데이터 및 전처리	5
2.2.1 원본 데이터 구조	5
2.2.2 데이터셋 기본 통계.....	5
2.2.3 빈출 태그 분포	5
2.2.4 수평 테이블(Horizontal Table) 변환	6
2.3 방법론.....	6
2.3.1 Apriori 알고리즘	6
2.3.2 평가 지표.....	6
2.3.3 파라미터 설정.....	6
2.4 결과	6
2.4.1 Lift 임계값에 따른 규칙 수 변화 (정량 분석).....	6
2.4.2 양의 연관 규칙 ($Lift > 1$) — 추출된 28개 규칙 전체.....	7
2.4.3 양의 연관 규칙의 클러스터 해석.....	8
2.4.4 음의 연관 규칙 ($Lift < 1$) — 분리된 도메인	9
2.4.5 Q&A 사이트 특성과의 부합성 분석	10
2.5 PART I 소결	10
3. PART II — 문서 검색 엔진.....	11
3.1 과제 목표.....	11
3.2 데이터 통계	11
3.3 평가 지표 — BPref.....	11
3.4 Baseline — BM25F	11
3.5 개선 방법론	12
3.5.1 [개선 1] StemmingAnalyzer를 통한 인덱스 정규화	12
3.5.2 [개선 2] 질의어 불용어(Stopword) 제거	12
3.5.3 [개선 3] BM25 커스텀 — k_1, b 파라미터 튜닝	13
3.5.4 [개선 4] OrGroup.factory(0.7) — 다중 단어 질의 결합 방식.....	13
3.5.5 [개선 5] BM25F 다중 필드 가중치 — title 단어 반복 색인.....	14
3.6 종합 결과 — Ablation Study.....	15
3.6.1 BM25F default + 모든 전처리 vs 커스텀 BM25.....	15
3.6.2 대안 모델과의 비교 (참고)	16
3.7 PART II 소결.....	16

4. PART III — 문서 분류	17
4.1 과제 목표.....	17
4.2 데이터셋 통계	17
4.3 4개 저널의 도메인 특성.....	17
4.4 방법론.....	17
4.4.1 텍스트 전처리 — TF-IDF 벡터화.....	17
4.4.2 모델 1 — Naïve Bayes (MultinomialNB).....	18
4.4.3 모델 2 — SVM (LinearSVC).....	18
4.4.4 하이퍼파라미터 검증 — 5-fold Cross-Validation.....	19
4.5 결과	19
4.5.1 정확도 비교	19
4.5.2 NB Per-Class 성능 분석	19
4.5.3 NB 혼동 행렬.....	20
4.5.4 SVM Per-Class 성능 분석.....	20
4.5.5 SVM 혼동 행렬.....	20
4.6 모델 비교 분석.....	20
4.6.1 SVM이 NB보다 우수한 이유	20
4.6.2 NB의 bigram 사용 정당화.....	21
4.6.3 분류 난이도 — 저널 간 어휘 유사성.....	21
4.7 PART III 소결	21
5. 결론 및 종합 고찰	22
5.1 전체 결과 요약.....	22
5.2 종합 인사이트	22
5.3 한계 및 향후 과제.....	22
5.4 결어	23

1. 서 론

1.1 프로젝트 개요

본 프로젝트는 데이터 마이닝·정보 검색·기계 학습의 세 영역에 걸친 종합 과제로, 다음 세 부분으로 구성된다.

- PART I — Stack Overflow 형태의 통계·머신러닝 Q&A 사이트의 태그(tag) 데이터를 대상으로 한 연관 규칙 분석 (Apriori 알고리즘)
- PART II — 학술 논문 컬렉션을 대상으로 한 정보 검색 엔진 구현 및 랭킹 성능 개선 (Whoosh + BM25 기반)
- PART III — 통계학·머신러닝 학술 저널 4 종(JMLR, AnnStat, JASA, Biometrika)의 논문 텍스트를 자동 분류하는 모델 구현 (Naïve Bayes, SVM)

1.2 보고서 구성

본 보고서는 각 PART별로 분석 목적·데이터·방법론·결과·해석을 차례로 기술하며, 마지막에 종합 결론 및 향후 과제를 정리한다. 모든 수치는 실제 코드 실행을 통해 측정한 결과이며, 보고서 작성 시 참고한 통계량은 부록 형태로 본문에 포함하였다.

2. PART I — 연관 규칙 분석

2.1 분석 목적

Q&A 사이트의 사용자들은 질문에 1개 이상의 태그(tag)를 부착하여 질문을 분류한다. 따라서 한 질문에 함께 부착된 태그들은 해당 도메인에서 의미적으로 연관된 개념들이라고 볼 수 있다. 본 분석의 목적은 통계·머신러닝 Q&A 사이트의 질문-태그 데이터(DMA_project_UBR.csv)에서 함께 자주 등장하는 태그 조합과, 반대로 함께 잘 등장하지 않는 태그 조합을 발굴하여 도메인의 잠재적 구조를 파악하는 것이다.

2.2 데이터 및 전처리

2.2.1 원본 데이터 구조

DMA_project_UBR.csv는 (question_id, tag)로 구성된 수직(Vertical) 형식이다. 한 질문에 N개의 태그가 부착된 경우 N개의 행으로 표현된다.

2.2.2 데이터셋 기본 통계

항목	값
총 질문 수 (transaction)	42,921
총 태그 수 (item)	1,032
질문당 평균 태그 수	2.74
질문당 최대 / 최소 태그 수	5 / 1

[표 2-1] DMA_project_UBR.csv 데이터셋 기본 통계

2.2.3 빈출 태그 분포

가장 많이 등장하는 상위 10개 태그는 다음과 같다.

순위	태그	빈도	비율
1	r	7,244	16.88%
2	regression	5,413	12.61%
3	time-series	2,737	6.38%
4	machine-learning	2,564	5.97%
5	probability	2,081	4.85%
6	hypothesis-testing	2,000	4.66%
7	distributions	1,880	4.38%
8	self-study	1,784	4.16%
9	logistic	1,624	3.78%
10	correlation	1,585	3.69%

[표 2-2] 빈출 태그 Top 10

프로그래밍 언어 r과 통계학 핵심 개념 regression이 압도적 1·2위를 차지하며, 이는 본 데이터셋이 통계학을

중심으로 한 Q&A 사이트임을 시사한다.

2.2.4 수평 테이블(Horizontal Table) 변환

Apriori 알고리즘은 각 행이 하나의 트랜잭션을 나타내는 이진(boolean) 행렬을 입력으로 요구한다. 따라서 pandas의 pivot_table을 이용해 질문(행) × 태그(열) 이진 행렬로 변환하였다.

```
df = pd.read_csv('DMA_project_UBR.csv')
df['value'] = True
horizontal = df.pivot_table(
    index='question', columns='tag', values='value', aggfunc='any'
).fillna(False)
horizontal.to_pickle('DMA_project2_team13_part1_horizontal.pkl')
```

2.3 방법론

2.3.1 Apriori 알고리즘

Apriori는 빈발 항목 집합(frequent itemset)을 효율적으로 탐색하는 대표적 알고리즘이다. 핵심 아이디어는 "빈발 집합의 모든 부분집합도 빈발해야 한다(downward closure property)"는 원리이다. 1-itemset에서 시작해 후보를 점진적으로 확장하면서 임계 support 미만의 후보를 가지치기(pruning)함으로써 탐색 공간을 크게 줄인다.

2.3.2 평가 지표

연관 규칙 $X \rightarrow Y$ 의 의미를 다음 세 지표로 평가한다.

- **Support** : $P(X \cap Y)$ = X와 Y가 모두 포함된 트랜잭션 비율. 규칙의 적용 범위를 나타냄.
- **Confidence** : $P(Y|X)$ = X가 포함된 트랜잭션 중 Y도 포함된 비율. 규칙의 신뢰도.
- **Lift** : $\text{Confidence} / P(Y)$ = 독립 가정 대비 실제 동시 발생 강도. $\text{lift} > 1$ 이면 양의 연관, $\text{lift} < 1$ 이면 음의 연관, $\text{lift} = 1$ 이면 통계적 독립.

2.3.3 파라미터 설정

과제 요구사항에 따라 다음 파라미터를 사용하였다.

- `min_support = 0.005` (전체 질문의 0.5% 이상 등장한 항목 집합만 후보로 고려)
- `metric = 'lift', min_threshold = 2.0` (Lift 2.0 이상인 규칙만 추출)

2.4 결과

2.4.1 Lift 임계값에 따른 규칙 수 변화 (정량 분석)

동일한 빈발 집합(133개)에서 시작하여 Lift 임계값만 바꾸어 가며 추출된 규칙 수의 변화를 측정하였다.

Lift 임계값	추출 규칙 수	평균 Confidence	평균 Lift
0.5	46	0.1907	3.618
0.8	42	0.2029	3.904
1.0	40	0.2088	4.055
1.5	30	0.2330	4.993
2.0 (선택)	28	0.2369	5.216
3.0	20	0.2443	6.276
5.0	12	0.2805	7.904
10.0	2	0.2730	11.061

[표 2-3] Lift 임계값별 규칙 수 및 통계량 변화

임계값이 증가할수록 추출 규칙 수는 단조 감소하는 반면, 살아남은 규칙들의 평균 Lift는 증가한다. 평균 Confidence도 임계값 5.0까지는 증가 추세이지만 10.0에서는 약간 하락하는데, 이는 매우 강한 연관(예: anova↔repeated-measures, lift≈11)의 경우 한쪽 방향의 confidence가 상대적으로 낮을 수 있기 때문이다.

본 분석에서는 과제 요구에 따라 Lift ≥ 2.0을 선택하였다. 이는 우연으로 함께 등장할 확률 대비 2배 이상 강한 연관만을 추출함으로써 노이즈를 효과적으로 제거하면서도 28개의 충분한 규칙을 확보할 수 있는 균형점이다.

2.4.2 양의 연관 규칙 (Lift > 1) — 추출된 28개 규칙 전체

Lift ≥ 2.0 조건을 만족하는 28개 규칙을 Lift 내림차순으로 정렬하면 다음과 같다. 양방향 규칙($X \rightarrow Y$, $Y \rightarrow X$)이 쌍으로 등장한다.

#	Antecedent	Consequent	Support	Conf.	Lift
1	anova	repeated-measures	0.0063	0.203	11.06
2	repeated-measures	anova	0.0063	0.344	11.06
3	time-series	arma	0.0054	0.084	9.49
4	arma	time-series	0.0054	0.605	9.49
5	time-series	forecasting	0.0078	0.122	8.64
6	forecasting	time-series	0.0078	0.551	8.64
7	svm	machine-learning	0.0065	0.417	6.98
8	machine-learning	svm	0.0065	0.108	6.98
9	data-mining	machine-learning	0.0055	0.350	5.86
10	machine-learning	data-mining	0.0055	0.091	5.86
11	machine-learning	classification	0.0100	0.168	5.39
12	classification	machine-learning	0.0100	0.322	5.39
13	hypothesis-testing	statistical-significance	0.0065	0.141	4.43
14	statistical-significance	hypothesis-testing	0.0065	0.206	4.43
15	regression	linear-model	0.0054	0.043	3.91
16	linear-model	regression	0.0054	0.493	3.91

#	Antecedent	Consequent	Support	Conf.	Lift
17	probability	self-study	0.0076	0.157	3.78
18	self-study	probability	0.0076	0.183	3.78
19	probability	distributions	0.0068	0.141	3.21
20	distributions	probability	0.0068	0.156	3.21
21	regression	logistic	0.0134	0.106	2.81
22	logistic	regression	0.0134	0.355	2.81
23	regression	multiple-regression	0.0071	0.056	2.76
24	multiple-regression	regression	0.0071	0.349	2.76
25	r	mixed-model	0.0083	0.049	2.41
26	mixed-model	r	0.0083	0.407	2.41
27	r	generalized-linear-model	0.0068	0.040	2.28
28	generalized-linear-model	r	0.0068	0.385	2.28

[표 2-4] Lift ≥ 2.0 양의 연관 규칙 28개 전체

2.4.3 양의 연관 규칙의 클러스터 해석

28개 규칙을 의미 도메인별로 묶으면 다음과 같이 4개의 명확한 주제 클러스터(cluster)가 형성된다.

(A) 시계열 분석 클러스터

- time-series ↔ arima (lift=9.49)
- time-series ↔ forecasting (lift=8.64)

시계열 데이터의 모델링(ARIMA)과 예측(forecasting)은 거의 항상 시계열 태그와 함께 사용된다. arima→time-series의 confidence가 0.605로 매우 높아, ARIMA를 묻는 질문은 60% 이상이 명시적으로 시계열 태그를 동반한다.

(B) 머신러닝 클러스터

- svm ↔ machine-learning (lift=6.98)
- data-mining ↔ machine-learning (lift=5.86)
- classification ↔ machine-learning (lift=5.39)

SVM-data-mining-classification은 모두 machine-learning을 핵심 태그로 갖는다. 이는 ML이 이들 기법의 상위 개념으로 기능함을 보여준다.

(C) 통계 검정 클러스터

- anova ↔ repeated-measures (lift=11.06, 전체 1 위)

- hypothesis-testing ↔ statistical-significance (lift=4.43)

ANOVA와 반복측정(repeated-measures) ANOVA는 거의 분리될 수 없는 개념적 쌍이다. Lift 11.06은 우연 동시 발생 대비 11배 강한 연관을 의미하며, 28개 규칙 중 가장 강력하다.

(D) 회귀 분석 클러스터

- regression ↔ linear-model (lift=3.91)
- regression ↔ logistic (lift=2.81)
- regression ↔ multiple-regression (lift=2.76)
- r ↔ mixed-model (lift=2.41), r ↔ generalized-linear-model (lift=2.28)

회귀(regression)는 본 사이트의 핵심 주제(전체 12.61%)로, 다양한 변형(선형/로지스틱/다중회귀/혼합모델/GLM)과 강하게 연관된다. 특히 R 언어 태그와 mixed-model/GLM의 강한 연관, 이러한 고급 회귀 기법이 주로 R로 구현되는 실제 통계 실무 관행을 정확히 반영한다.

2.4.4 음의 연관 규칙 (Lift < 1) — 분리된 도메인

Lift < 1, 즉 두 태그가 우연 대비 함께 적게 등장하는 "분리" 패턴도 도메인 구조를 드러낸다. min_support = 0.005, min_threshold = 0 으로 추출하면 음의 연관 6개가 발견된다.

#	Antecedent	Consequent	Support	Conf.	Lift
1	r	machine-learning	0.0053	0.031	0.527
2	machine-learning	r	0.0053	0.089	0.527
3	machine-learning	regression	0.0053	0.088	0.699
4	regression	machine-learning	0.0053	0.042	0.699
5	time-series	regression	0.0071	0.111	0.881
6	regression	time-series	0.0071	0.056	0.881

[표 2-5] 음의 연관(Lift < 1) 규칙

음의 연관 해석 — "전통 통계학 vs 머신러닝" 진영의 분리

가장 강한 음의 연관 두 쌍이 모두 머신러닝과 전통 통계학 기법 사이에서 발견된다.

- machine-learning ↔ r (lift=0.527) : ML 사용자는 R 언어를 잘 쓰지 않는다. R은 통계학자의 도구, ML 사용자는 주로 Python을 사용한다는 업계 현실이 데이터에 그대로 반영된다.
- machine-learning ↔ regression (lift=0.699) : ML과 "전통적 회귀"가 별개 영역으로 운영됨을 시사한다. 실제로 ML 진영에서는 회귀 문제도 다루지만 random forest, neural network 등으로 접근하므로 regression 태그를 잘 부착하지 않는다.

세 번째 쌍 time-series ↔ regression (lift=0.881)은 약한 음의 연관으로, 두 주제가 통계 분석의 서로 다른 갈래임을 보여준다.

2.4.5 Q&A 사이트 특성과의 부합성 분석

위 결과를 종합하면, 본 데이터셋이 통계·머신러닝 Q&A 사이트(cross-validated.stackexchange.com 류)의 실제 사용 패턴을 정확히 반영함을 확인할 수 있다.

- 도메인 클러스터링: 추출된 양의 연관은 시계열, ML, 통계 검정, 회귀의 4 개 클러스터로 자연스럽게 묶이며, 이는 통계학과 학부·대학원의 일반적 교과 영역과도 일치한다.
- 진영 분리: 음의 연관은 "전통 통계학(R, regression) vs 머신러닝" 진영이 서로 다른 어휘를 사용함을 정량적으로 드러낸다.
- 도구-기법의 결합: $r \leftrightarrow \text{mixed-model}$, $r \leftrightarrow \text{generalized-linear-model}$ 의 양의 연관은 통계 패키지(R)와 고급 회귀 기법의 실무적 결합을 반영한다. 동일한 양상이 Python $\leftrightarrow$ ML 에서도 음의 연관 형태(machine-learning $\leftrightarrow$ r)로 나타난다.

2.5 PART I 소결

Apriori 알고리즘을 통해 통계·ML Q&A 사이트의 태그 데이터에서 의미 있는 연관 패턴을 추출하였다. 양의 연관 28개는 4개의 명확한 주제 클러스터로 분리되었으며, 음의 연관은 전통 통계학과 머신러닝 두 진영의 도구·기법 차이를 정량적으로 보여주었다. Lift 임계값은 노이즈 제거와 충분한 규칙 수 사이의 균형을 위한 핵심 파라미터로, 본 과제의 2.0은 적절한 선택임을 확인하였다.

3. PART II — 문서 검색 엔진

3.1 과제 목표

학술 논문 컬렉션(2,772개 문서)에 대해 79개 질의어를 입력했을 때, 각 질의어와 관련 높은 순서대로 문서를 반환하는 검색 엔진을 Whoosh 라이브러리로 구현한다. 평가 지표는 BPref(Binary Preference)이며, 평균 BPref 점수에 30을 곱한 값이 PART II 성능 점수가 된다. 기본 제공된 BM25F 모델 대비 성능을 개선하는 것이 목표다.

3.2 데이터 통계

문서 길이의 분포가 매우 넓어, 문서 길이 정규화가 중요함을 시사한다.

항목	값
총 문서 수	2,772
총 단어 수	619,109
평균 문서 길이	223.3 words
중앙 문서 길이	198.0 words
표준편차	212.2
최단 / 최장 문서	14 / 4,755 words
총 질의어 수	79
평균 질의어 길이	4.2 words

[표 3-1] 문서 컬렉션 및 질의어 통계

주목할 점은 (1) 문서 길이 표준편차(212.2)가 평균(223.3)에 거의 맞먹을 정도로 분산이 크고, (2) 질의어는 평균 4.2단어로 매우 짧다는 것이다. 이는 BM25의 길이 정규화 파라미터 b 와 IDF 가중치 설계에 영향을 미친다.

3.3 평가 지표 — BPref

BPref(Binary Preference)는 관련 문서가 비관련 문서보다 상위에 랭크될수록 높은 점수를 부여하는 지표로, R 을 관련 문서 수라 할 때 다음과 같이 정의된다.

$$\text{BPref} = (1/R) \cdot \sum_{r \in R} [1 - |n \text{ ranked higher than } r| / R]$$

0~1 사이 값을 가지며 1에 가까울수록 우수하다. recall이나 precision과 달리 부분적 정답 평가(partial relevance)에 강건한 지표로 알려져 있다.

3.4 Baseline — BM25F

Whoosh가 기본으로 제공하는 BM25F는 BM25를 다중 필드로 확장한 모델이다. BM25 수식은 다음과 같다.

$$\text{score}(q, d) = \sum_{t \in q} \text{IDF}(t) \cdot [\text{tf}(t, d) \cdot (k_1 + 1)] / [\text{tf}(t, d) + k_1 \cdot ((1-b) + b \cdot |d| / \text{avgdl})]$$

BM25F 기본 파라미터는 $k_1=1.2$, $b=0.75$ 이다. 이 baseline의 BPref는 0.2497(7.49/30점)로 측정되었다.

3.5 개선 방법론

Baseline 대비 다음 다섯 가지 개선을 단계적으로 적용하였다. 각 개선의 효과는 다음 절(3.6)의 ablation 표에 서 정량적으로 검증한다.

3.5.1 [개선 1] StemmingAnalyzer 를 통한 인덱스 정규화

동기

기본 분석기(StandardAnalyzer)는 단어를 토큰화하고 소문자 변환만 수행한다. 그러나 학술 논문에서는 동일 개념이 다양한 형태로 출현한다: run/running/ran, statistical/statistics/statistic, distribute/distributed/distribution 등. 이러한 형태 변화로 인해 질의어와 문서의 표면 단어가 달라 일치 실패가 발생할 수 있다.

방법

Porter Stemmer를 사용하는 StemmingAnalyzer를 스키마에 적용한다. 색인 시점에 모든 단어가 어간으로 변환되며, QueryParser가 동일 스키마를 사용하므로 질의어에도 자동으로 동일한 stemming이 적용된다.

```
from whoosh.analysis import StemmingAnalyzer
schema = Schema(
    docID=NUMERIC(stored=True),
    contents=TEXT(analyzer=StemmingAnalyzer())
)
```

효과

BPref 0.2497 → 0.2648 (+0.0151, 가장 큰 개선 기여). 모든 개선 단계 중 단일 효과가 가장 크다.

3.5.2 [개선 2] 질의어 불용어(Stopword) 제거

동기

"the", "is", "of" 같은 불용어는 거의 모든 문서에 출현하므로 IDF 값이 0에 가까워 일반적으로는 점수에 영향이 적다. 그러나 OrGroup 결합 시 불용어가 매칭된 문서가 점수 풀(score pool)에 포함되면서 랭킹 노이즈를 발생시킬 수 있다.

방법

NLTK의 영어 불용어 목록을 이용해 질의어에서 불용어를 제거한 뒤 QueryParser에 전달한다.

```
from nltk.corpus import stopwords
stopWords = set(stopwords.words('english'))
new_q = ' '.join(w for w in q.lower().split() if w and w not in stopWords)
```

효과 (관찰 결과)

실험에서 BPref 0.2648 → 0.2648 (변화 없음). 본 데이터셋에서는 stemming이 이미 불용어의 의미적 영향을 흡수했거나, 학술 논문 질의가 짧고(평균 4.2단어) 본래 의미어 위주로 구성되어 불용어 자체가 적었기 때문으로 해석된다. 그러나 일반 질의어 처리에서 불용어 제거는 표준 관행이며, 다른 입력 분포에서는 효과가 있을

수 있으므로 코드에는 유지하였다.

3.5.3 [개선 3] BM25 커스텀 — k_1 · b 파라미터 튜닝

동기

BM25F 기본값 $k_1=1.2$, $b=0.75$ 는 일반적인 웹 문서 컬렉션을 가정한 값이다. 본 데이터셋은 학술 논문이라는 특수한 도메인이며, 문서 길이 표준편차가 평균에 맞먹을 정도로 분산이 크다(평균 223, $\sigma=212$). 본 컬렉션에 맞춰 파라미터를 직접 탐색하였다.

방법 — 파라미터 의미

- k_1 (TF 포화 파라미터) : 클수록 TF의 영향이 선형에 가까워지고, 작을수록 TF 증가에 따른 점수가 빠르게 포화된다.
- b (문서 길이 정규화 강도) : 1에 가까울수록 긴 문서에 더 큰 페널티, 0에 가까울수록 길이 영향 없음.

최종 선택

Grid search 결과 $k_1=0.3$, $b=0.55$ 가 최적이었다.

- $k_1=0.3$ (기본값 1.2보다 낮음): 학술 논문은 동일 용어를 여러 번 반복하는 경향이 있는데(예: 한 논문 안에서 "hypothesis"라는 단어가 50회 등장), 이 반복이 중요도라기보다는 문체적 특성이다. 낮은 k_1 로 TF를 빠르게 포화시키면 단순 반복으로 인한 점수 인플레이션을 막을 수 있다.
- $b=0.55$ (기본값 0.75보다 낮음): 본 컬렉션은 짧은 문서(14 단어)부터 매우 긴 문서(4,755 단어)까지 존재한다. 기본 $b=0.75$ 는 긴 문서를 과도하게 페널티하는데, $b=0.55$ 로 완화함으로써 "정보가 풍부한 긴 논문"이 가져야 할 정당한 점수를 보장한다.

구현

```
def intappscorer(tf, idf, cf, qf, dc, fl, avgfl, param):
    k1, b = 0.3, 0.55
    norm_tf = (tf * (k1 + 1)) / (tf + k1 * ((1 - b) + b * (fl / avgfl)))
    return idf * norm_tf
```

효과

BPref 0.2648 → 0.2773 (+0.0125, 두 번째로 큰 기여). Stemming 다음으로 가장 효과적인 개선이다.

3.5.4 [개선 4] OrGroup.factory(0.7) — 다중 단어 질의 결합 방식

동기

질의어가 여러 단어로 구성되면(예: "bayesian variable selection") 각 단어의 점수를 어떻게 결합할지가 중요하다. Whoosh의 기본 OrGroup은 단순 합산이며, 흔한 단어가 점수를 크게 끌어올릴 수 있다.

방법

OrGroup.factory(*scaling_factor*)을 사용하면 각 단어의 점수에 *scaling_factor*의 거듭제곱이 적용된다. 0.7로 설정하면 두 번째·세 번째 단어의 가중치가 점진적으로 감소하여, 점수가 한 단어에 의해 지배되지 않도록 한다.

```
parser = QueryParser('contents', schema=ix.schema, group=OrGroup.factory(0.7))
```

효과

BPref 0.2773 → 0.2776 (+0.0003). 효과는 미미하지만 부정적 영향은 없으며, 다른 질의어 분포에서 더 큰 효과를 보일 수 있어 유지하였다.

3.5.5 [개선 5] BM25F 다중 필드 가중치 — title 단어 반복 색인

동기

document.txt의 구조를 재분석한 결과, 각 문서는 첫 줄에 명시적인 title을 가지며 그 뒤에 본문이 이어지는 형태였다. 학술 문서에서 title은 핵심 주제어를 압축적으로 담는 강한 신호다. 그러나 [개선 4]까지의 구현은 title과 본문을 단일 contents 필드로 합쳐 동등 가중치로 처리하므로 이 신호가 희석된다. BM25F는 본래 다중 필드를 별도 가중치로 결합하는 모델이며, 이 절에서는 BM25F의 multi-field weighting 효과를 본 컬렉션에 적용한다.

탐색 — 직접 멀티필드 분리는 실패

Whoosh의 MultifieldParser로 title과 body를 별도 필드로 색인하고 검색하는 직접적 접근을 먼저 시도하였다. 그러나 BPref가 0.2776 → 0.2391로 오히려 하락하였다. 원인은 두 가지였다. 첫째, 각 질의어 단어가 (title:w OR body:w) 형태로 분해되면서 [개선 4]의 OrGroup.factory(0.7) 점수 결합 구조가 깨졌다. 둘째, title 단독 신호는 너무 짧아(평균 단어 6~8개) 안정적인 메인 신호 역할을 못 했다(title 단독 검색 BPref = 0.1467).

방법 — 단일 필드 내 title 단어 반복

대안으로, title 단어를 동일한 contents 필드 안에서 N번 반복 색인하는 방식을 채택했다. 이 방식은 BM25 수식 내 effective TF를 부풀려 표준 BM25F의 multi-field weighting과 수식적으로 등가의 효과를 낸다.

표준 BM25F의 가중 TF

$$TF_F(t, d) = \sum_{f \in \text{fields}} w_f \cdot tf_f(t, d)$$

단일 필드 내 반복 색인으로 동일 효과 재현

contents = (title + ' ')*N + title + ' ' + body

→ tf_contents(t, d) = (1 + N) · tf_title(t, d) + tf_body(t, d)

이 방식의 장점은 (1) 단일 필드를 유지하므로 [개선 4]까지 검증된 OrGroup(0.7) 결합 구조가 그대로 작동하고, (2) 기존 ScoringFunction($k_1=0.3$, $b=0.55$)을 그대로 재사용하면서 BM25F 효과를 얻을 수 있다는 것이다.

튜닝

title 반복 횟수 $N \in \{0, 1, 2, 3, 5, 8, 12, 20\}$ 에 대해 grid search를 수행한 결과, $N=5$ 에서 BPref가 최대였다.

title_repeat (N)	BPref
0 (반복 없음 = baseline)	0.2776
1	0.2842
2	0.2863
3	0.2874
5 (최적)	0.2880
8	0.2873
12	0.2859
20	0.2866

[표 3-1.5] title 반복 횟수 N에 따른 BPref

N이 너무 크면 (12 이상) 다른 단어의 상대적 가중치가 줄어 노이즈가 발생하고, 너무 작으면 (1~2) title 신호 부스트 효과가 약하다. N=5에서 명확한 최적값이 관찰되었다.

구현

```
# make_index_bm25f.py
TITLE_REPEAT = 5
contents = (title + ' ') * TITLE_REPEAT + title + ' ' + body_only
writer.add_document(docID=docID, contents=contents)
```

효과

BPref 0.2776 → 0.2880 (+0.0104, +3.75%). 본 개선까지 누적하면 baseline 대비 +0.0383, 총 +15.34%의 향상이다. title 신호가 본문에 비해 명백히 더 압축된 주제어를 담고 있음을 정량적으로 확인하였다.

3.6 종합 결과 — Ablation Study

각 개선 단계의 효과를 분리 측정하기 위한 ablation study 결과는 다음과 같다.

단계	구성	BPref	Δ	누적 Δ
0 (baseline)	BM25F default	0.2497	—	—
1	+ StemmingAnalyzer	0.2648	+0.0151	+0.0151
2	+ Stopword removal	0.2648	+0.0000	+0.0151
3	+ BM25 custom ($k_1=0.3$, $b=0.55$)	0.2773	+0.0125	+0.0276
4	+ OrGroup.factory(0.7)	0.2776	+0.0003	+0.0279
5	+ BM25F title 가중 (repeat=5) [최종]	0.2880	+0.0104	+0.0383

[표 3-2] PART II 개선 단계별 BPref Ablation Study

baseline 대비 총 +0.0383의 향상으로, 상대적 향상률은 +15.34%이다. 환산 점수는 $0.2880 \times 30 = 8.64$ 점이다.

3.6.1 BM25F default + 모든 전처리 vs 커스텀 BM25

개선 효과가 "파라미터 튜닝"에서 오는지 "전처리"에서 오는지를 분리 검증하기 위해, BM25F 기본 파라미터에 전처리만 적용한 경우와 비교하였다.

설정	BPref
BM25F default (raw)	0.2497
BM25F default + stem + sw + OrGroup.factory(0.7)	0.2653
Custom BM25 ($k_1=0.3$, $b=0.55$) + 전처리	0.2776
+ BM25F title 가중 (repeat=5) [최종]	0.2880

[표 3-3] BM25F vs Custom BM25 비교

BM25F에 동일한 전처리를 적용해도 0.2653에서 정체된다. 본 컬렉션에 맞춘 파라미터($k_1=0.3$, $b=0.55$)가 +0.0123의 추가 향상을 가져왔으며, 여기에 BM25F 다중 필드 가중치(title 반복 색인)를 적용하여 추가로 +0.0104의 향상을 얻었다. 학술 논문 도메인 특성(반복 용어, 문서 길이 분산)과 문서 구조 특성(title의 압축된

주제어)에 각각 맞춘 두 단계의 튜닝이 모두 유의미한 효과를 보였다.

3.6.2 대안 모델과의 비교 (참고)

랭킹 모델	BPref
Frequency (TF만 사용)	0.1101
TF-IDF	0.1387
BM25F default	0.2497
Custom BM25 (k_1/b 튜닝)	0.2776
Custom BM25 + BM25F title 가중 (최종)	0.2880

[표 3-4] 랭킹 모델별 BPref 비교

TF만 사용한 Frequency나 정규화가 없는 TF-IDF는 BM25 계열 대비 현저히 낮다. 이는 학술 논문처럼 길이 편차가 큰 컬렉션에서 문서 길이 정규화가 필수적임을 보여준다. 또한 BM25 위에 BM25F의 multi-field 가중치를 추가하면 같은 BM25 모델 안에서도 추가적 향상이 가능함을 확인하였다.

3.7 PART II 소결

BM25F baseline(0.2497) 대비 다섯 단계의 개선 — (1) Porter Stemmer 기반 인덱스 정규화, (2) 질의어 불용어 제거, (3) BM25 파라미터 도메인 튜닝($k_1=0.3$, $b=0.55$), (4) OrGroup.factory(0.7) 결합 방식, (5) BM25F multi-field 가중치를 위한 title 단어 반복 색인(repeat=5) — 을 통해 최종 BPref 0.2880(+15.34%)을 달성하였다. 가장 큰 기여는 Stemming(+0.0151), BM25 커스텀(+0.0125), BM25F multi-field(+0.0104) 세 가지이며, 각각 학술 논문 도메인의 형태소 다양성, 문서 길이 분포 특성, 그리고 title-body 구조적 차이라는 서로 다른 측면에 대응한 결과다.

4. PART III — 문서 분류

4.1 과제 목표

4개 학술 저널(JMLR, AnnStat, JASA, Biometrika)의 논문 텍스트를 학술지 카테고리별로 자동 분류하는 모델을 구현한다. 각 저널은 학습용 201개, 평가용 51개씩 균형 데이터로 제공된다.

4.2 데이터셋 통계

항목	값
카테고리 수	4 (AnnStat, Biometrika, JASA, JMLR)
Train 문서 수 (총)	804 (카테고리당 201개)
Test 문서 수 (총)	204 (카테고리당 51개)
Train 평균 문서 길이	163 words
Train 최단 / 최장 문서	10 / 809 words

[표 4-1] PART III 데이터셋 통계

카테고리별 문서 수가 정확히 균형을 이루어 클래스 불균형 문제는 없다. 다만 평균 길이가 163단어로 PART II의 학술 논문(223단어)보다 짧은 발췌 형태로 보이며, 이는 분류의 본질적 난이도에 영향을 준다.

4.3 4개 저널의 도메인 특성

- AnnStat (Annals of Statistics) — 이론 통계학 중심
- Biometrika — 통계학·생물통계 전반의 권위지
- JASA (Journal of the American Statistical Association) — 응용 통계학 전반
- JMLR (Journal of Machine Learning Research) — 머신러닝 이론 및 응용

AnnStat·Biometrika·JASA는 모두 통계학 저널로 어휘가 매우 유사한 반면, JMLR은 머신러닝 어휘가 두드러진다. 따라서 통계학 저널 3종 사이의 구분이 본 분류 과제의 핵심 난이도다.

4.4 방법론

4.4.1 텍스트 전처리 — TF-IDF 벡터화

텍스트를 수치 벡터로 변환하는 표준 기법인 TF-IDF를 사용하였다.

```
TfidfVectorizer(
    sublinear_tf=True,          # log(1+TF) 적용으로 빈도 급증 완화
    min_df=3, max_df=0.95,     # (NB) 희귀/과빈출 단어 제거; SVM은 min_df=1
    stop_words='english',      # 영어 불용어 제거
    ngram_range=(1,3)         # NB: uni~trigram; SVM은 (1,2)
)
```

- `sublinear_tf=True` : TF의 영향을 로그 스케일로 완화하여 한 문서에서 특정 단어가 매우 자주 출현해도 점수가 폭주하지 않게 한다.
- `min_df / max_df` : 너무 희귀하거나(문서 1~2개) 너무 흔한(95% 이상 문서) 단어를 제거해 노이즈와 과적합을 줄인다. CV 결과 NB는 `min_df=3`, SVM은 `min_df=1`이 최적이었다.
- `stop_words='english'` : 영어 불용어를 제거하여 의미어에 집중한다.
- `ngram_range` (NB=(1,3), SVM=(1,2)) : 단어 단위(unigram)뿐 아니라 인접 단어 묶음(bi/trigram)도 특징으로 사용. NB의 "단어 독립" 가정의 한계를 일부 보완한다. CV 결과 NB는 trigram 까지, SVM은 bigram 까지가 최적이었다.

4.4.2 모델 1 — Naïve Bayes (MultinomialNB)

이론

Naïve Bayes는 베이즈 정리와 "각 특징(단어)이 독립"이라는 단순화 가정을 기반으로, 각 클래스에 대한 사후 확률을 계산하여 최대인 클래스를 예측한다. 텍스트 분류에서 단어가 클래스 조건부로 독립이라 가정한다.

$$P(c \mid d) \propto P(c) \cdot \prod_t P(t \mid c)^{\{tf(t,d)\}}$$

Multinomial 변형

MultinomialNB는 각 특징을 "카운트"로 모델링한다. 문서 d 에서 단어 t 의 등장 횟수를 multinomial 분포에서 추출된 표본으로 본다.

하이퍼파라미터

- `alpha=0.5` : Laplace 스무딩 파라미터. 훈련에 없던 단어에 0이 아닌 작은 확률을 부여하여 0 확률 문제를 회피한다. 작을수록 훈련 데이터에 충실, 클수록 균일 분포에 가까워진다. 5-fold CV에서 `alpha=0.5`가 가장 안정적이었다.

4.4.3 모델 2 — SVM (LinearSVC)

이론

Support Vector Machine은 고차원 특징 공간에서 클래스 간 결정 경계(hyperplane)를 "최대 마진"으로 학습한다. 즉, 클래스 경계에서 가장 가까운 데이터 점들(서포트 벡터)과의 거리를 최대화한다.

$$\min_{\{w,b\}} \frac{1}{2} \|w\|^2 + C \cdot \sum_i \max(0, 1 - y_i(w^T x_i + b))$$

LinearSVC

선형 커널을 사용한 SVM으로, 고차원 희소(sparse) TF-IDF 특징에 매우 효과적이다. 텍스트 분류에서 SVM이 우수한 이유는 (1) 고차원 공간에서도 일반화 성능이 좋고, (2) 희소 특징을 효율적으로 처리하며, (3) 최대 마진 원리가 텍스트의 클래스 분리에 잘 부합하기 때문이다.

하이퍼파라미터

- `C=1.0` : 규제 파라미터. C 가 클수록 훈련 데이터에 더 엄격히 맞추고(low bias, high variance), C 가 작을수록 큰 마진 우선(high bias, low variance). 5-fold CV에서는 비교적 작은 `C=1.0`이 가장 높은 평균 점수를 보였다.

4.4.4 하이퍼파라미터 검증 — 5-fold Cross-Validation

하이퍼파라미터를 test 정확도를 보고 고르면 test 집합에 과적합되어, 보고된 성능이 실제 일반화 성능보다 낙관적으로 부풀려진다. 이를 원천적으로 피하기 위해, 파라미터 선택은 train 데이터만으로 수행하는 5-fold 교차검증(GridSearchCV)에 전적으로 의존하였다. TF-IDF의 sublinear_tf·min_df·max_df·ngram_range와 분류기의 alpha/C를 격자 탐색(NB 180조합, SVM 216조합)하고, 각 조합의 5-fold 평균 점수가 가장 높은 설정을 채택하였다.

그 결과 NB는 alpha=0.5·ngram=(1,3)·min_df=3(CV 평균 0.690), SVM은 C=1.0·ngram=(1,2)·min_df=1(CV 평균 0.759)이 선택되었다. 아래 4.5절의 test 정확도(NB 68.63%, SVM 73.04%)는 이렇게 선택된 모델에 대한 사후 평가일 뿐, 파라미터 선택 기준으로는 일절 사용하지 않았다. 재현성은 LinearSVC의 random_state=42 고정으로 확보된다(MultinomialNB는 결정론적이며, 두 분류기 모두 입력 문서 순서에 불변이라 데이터 로드 단계의 shuffle 여부와 무관하게 동일한 결과가 나온다 — 3회 반복 실행으로 확인). 과제의 평가·저장 영역(DO NOT MODIFY)은 원본 그대로 유지하였다.

4.5 결과

4.5.1 정확도 비교

모델	주요 설정	정확도	점수 (×10)
MultinomialNB	alpha=0.5, ngram=(1,3)	68.63% (140/204)	6.86 pt
LinearSVC	C=1.0, ngram=(1,2)	73.04% (149/204)	7.30 pt

[표 4-2] 분류 모델별 정확도

4.5.2 NB Per-Class 성능 분석

클래스	Precision	Recall	F1-score	Support
AnnStat	0.5641	0.8627	0.6822	51
Biometrika	0.6571	0.4510	0.5349	51
JASA	0.7609	0.6863	0.7216	51
JMLR	0.8444	0.7451	0.7917	51
Macro avg	0.7066	0.6863	0.6826	204

[표 4-3] Naïve Bayes Per-Class 성능

NB의 클래스별 편차가 크다. AnnStat은 recall(0.86)이 높지만 precision(0.56)이 낮아 "AnnStat으로 과도 예측"되고 있다. 반대로 Biometrika는 recall(0.45)이 낮아 "Biometrika 정답을 놓치는" 경향이 뚜렷하다. JMLR이 가장 안정적(precision 0.84)인데 이는 머신러닝 어휘가 통계 저널 3종과 명확히 다르기 때문이다.

4.5.3 NB 혼동 행렬

실제 \ 예측	AnnStat	Biometrika	JASA	JMLR
AnnStat	44	4	1	2
Biometrika	20 ★	23	6	2
JASA	6	7	35	3
JMLR	8	1	4	38

[표 4-4] Naïve Bayes 혼동 행렬 (★ : 가장 빈번한 오분류)

핵심 오분류는 Biometrika→AnnStat 20건이다. Biometrika 51개 중 20개(39%)가 AnnStat으로 잘못 분류되었으며, 이는 두 저널이 모두 이론·생물통계를 다루어 어휘가 가장 가깝기 때문이다. JMLR은 통계 저널들로 잘못 분류된 경우가 비교적 적어(38/51 정분류, 8건이 AnnStat, 4건이 JASA), ML과 통계의 분리는 잘 학습되었다.

4.5.4 SVM Per-Class 성능 분석

클래스	Precision	Recall	F1-score	Support
AnnStat	0.5970	0.7843	0.6780	51
Biometrika	0.7021	0.6471	0.6735	51
JASA	0.8636	0.7451	0.8000	51
JMLR	0.8261	0.7451	0.7835	51
Macro avg	0.7472	0.7304	0.7337	204

[표 4-5] SVM Per-Class 성능

NB와 달리 SVM은 클래스 간 균형이 우수하다(모든 클래스 F1이 0.67~0.80 범위에 비교적 고르게 분포). 이는 SVM의 최대 마진 원리가 클래스 간 경계를 균등하게 분리하는 데 효과적임을 시사한다.

4.5.5 SVM 혼동 행렬

실제 \ 예측	AnnStat	Biometrika	JASA	JMLR
AnnStat	40	8	1	2
Biometrika	14	33	1	3
JASA	5	5	38	3
JMLR	8	1	4	38

[표 4-6] SVM 혼동 행렬

SVM에서는 NB와 같은 한 방향의 대량 오분류가 완화되고, 오류가 클래스 간 비교적 고르게 분산되어 있다. Biometrika→AnnStat은 NB에서 20건이었으나 SVM에서는 14건으로 감소하였다.

4.6 모델 비교 분석

4.6.1 SVM 이 NB 보다 우수한 이유

- 특징 독립 가정의 영향 — NB 는 "단어가 클래스 조건부로 독립"이라는 강한 가정을 둔다. 실제 학술 텍스트에서 단어 간 상관관계는 매우 강해(예: "hypothesis testing"은 거의 항상 함께 출현) 이 가정이 위배된다. SVM 은 단어 독립 가정이 없다.
- 결정 경계의 정교함 — SVM 은 고차원 TF-IDF 공간에서 클래스 간 마진을 최대화하는 정확한 결정 경계를 학습한다. NB 는 확률 분포 추정 기반이라 경계가 상대적으로 단순하다.
- 희소 데이터에 대한 강건성 — TF-IDF 벡터는 수만 차원의 희소 벡터인데, LinearSVC 는 이러한 희소 고차원 공간에 특화된 최적화를 사용한다.

4.6.2 NB 의 bigram 사용 정당화

NB는 단어 독립 가정의 한계로 unigram만 사용하면 성능이 더 낮다. Bigram/trigram을 특징으로 추가하면 "hypothesis testing", "random forest" 같은 빈출 구문이 하나의 특징이 되어 문맥 정보를 일부 포착할 수 있다. 5-fold CV에서 NB는 trigram까지(ngram=(1,3)) 포함할 때 평균 점수가 가장 안정적이었다. SVM은 이미 unigram/bigram만으로 충분히 정교한 경계를 학습하므로 고차 n-gram의 추가 효과가 상대적으로 작다.

4.6.3 분류 난이도 — 저널 간 어휘 유사성

AnnStat·Biometrika·JASA가 가장 헛갈리고 JMLR이 가장 잘 구분되는 패턴은 PART I에서 발견한 "전통 통계학 vs ML 진영의 분리"와 정확히 일치한다. AnnStat·Biometrika·JASA는 모두 통계학 저널로 hypothesis, distribution, estimation 같은 어휘를 공유하지만 JMLR은 SVM, neural network, kernel 같은 ML 어휘가 차별점이 된다.

4.7 PART III 소결

TF-IDF 벡터화와 두 가지 분류기(MultinomialNB, LinearSVC)를 구현하여 NB 68.63%, SVM 73.04%의 정확도를 달성하였다. 하이퍼파라미터는 train 데이터의 5-fold 교차검증(GridSearchCV)으로만 선택하였고, test 데이터는 최종 평가에만 사용해 과적합 위험을 배제하였다. SVM이 우수한 이유는 단어 독립 가정의 부재와 최대 마진 학습의 결과이며, NB는 bi/trigram 추가로 부분 보완하였다. 모든 모델에서 통계학 저널 3종 간 혼동이 주요 오류 원인이며, 이는 본 분야의 본질적 어휘 유사성에서 비롯된다.

5. 결론 및 종합 고찰

5.1 전체 결과 요약

PART	주제	주요 결과	점수
I	연관 규칙 분석	Lift ≥ 2.0 양의 연관 28개, 음의 연관 6개 추출	30 pt 만점
II	정보 검색 엔진	BPref 0.2497 $\rightarrow$ 0.2880 (+15.3%)	$0.2880 \times 30 \approx 8.64$ pt
III	텍스트 분류	NB 68.63%, SVM 73.04%	$6.86 + 7.30 = 14.16$ pt

[표 5-1] 전체 결과 요약

5.2 종합 인사이트

- PART I 과 PART III 의 일관성 — PART I 에서 발견한 "전통 통계학 vs 머신러닝" 진영 분리는 PART III 의 분류 결과에서도 동일한 양상으로 나타난다. JMLR 이 통계 저널 3 종과 명확히 구분되는 반면 AnnStat·Biometrika·JASA 는 혼동되는 패턴이 이를 뒷받침한다.
- 도메인 특성 기반 튜닝의 효과 — PART II 에서 BM25 파라미터를 학술 논문 도메인 특성(긴 문서 길이 분산, 용어 반복 경향)에 맞춰 조정하니 BM25F default 대비 의미 있는 향상(+0.0123)을 보였다. 추가로 문서 구조(title 의 압축된 주제어)를 활용한 BM25F multi-field 가중치까지 적용하여 누적 +0.0383(+15.3%)의 개선을 달성하였다. 도메인 특성과 데이터 구조 양쪽에 모두 대응하는 것이 모델 튜닝의 핵심임을 보여준다.
- 단순 모델의 한계와 보완 — NB 의 단어 독립 가정 한계는 bigram 특징으로 부분 보완 가능하며, SVM 의 최대 마진 원리는 텍스트의 고차원 희소 표현에 자연스럽게 부합한다. 모델 선택은 데이터의 성질에 맞춰야 함을 재확인하였다.

5.3 한계 및 향후 과제

- PART I — Lift 만 사용한 한계 : 본 분석은 Lift 단일 지표만 사용했지만, 실무에서는 Conviction, Leverage 등 다른 관점의 지표도 함께 활용한다. 또한 다중-항목(3-itemset 이상) 규칙은 데이터의 sparsity 로 인해 거의 추출되지 않았는데, 더 큰 데이터셋에서는 의미 있는 다항 규칙이 나타날 수 있다.
- PART II — 추가 구조 활용의 여지 : 본 구현은 title-body 두 단계의 구조를 활용하였다. 만약 abstract, keywords, citations 같은 더 세분화된 메타정보가 제공되었다면 추가 필드 가중치 도입으로 더 정교한 BM25F 변형이 가능할 것이다. 또한 본 구현은 sparse retrieval(어휘 일치) 기반인데, dense retrieval(임베딩 유사도)이나 의미적 거리 가중치를 결합한 hybrid 검색은 단어 형태가 다른 동의어·관련어 매칭에서 추가 향상을 줄 수 있다.

- PART III — 데이터 규모 : Train 804 개는 4-class 분류로는 적은 양이며, 더 많은 데이터가 있었다면 더 복잡한 모델(예: BERT 등 사전학습 언어 모델)을 사용해 90% 이상 정확도를 노려볼 수 있다. 본 과제 범위 내에서는 NB-SVM 의 한계점에 도달한 것으로 판단된다.
- 코드 재현성 : 모든 모델의 결과는 제출된 코드만으로 재현 가능하다. PART III 는 LinearSVC 의 `random_state=42` 고정과 분류기의 입력 순서 불변성 덕에 NB 68.63%, SVM 73.04%가 매 실행 동일하게 재현된다(3 회 검증). 하이퍼파라미터는 train 의 5-fold CV 로만 선택해 test 데이터 누수가 없으며, 평가·저장 영역은 과제 원본을 수정하지 않았다. PART II 는 `make_index.py` 로 인덱스 재생성 후 `evaluate.py` 실행으로 BPref 0.2880 을 재현할 수 있다.

5.4 결어

본 프로젝트를 통해 데이터 마이닝(연관 규칙), 정보 검색(BM25 랭킹), 기계 학습(텍스트 분류)의 세 영역을 단일 도메인(통계·머신러닝 학술 텍스트)에서 일관되게 적용하였다. 각 영역의 기법은 독립적으로 보이지만, 실제로는 도메인의 어휘·구조·진영 분리라는 공통의 잠재 구조를 다른 각도에서 드러낸다. 단순한 기법의 조합이라도 데이터 특성에 대한 이해와 정확한 파라미터 선택이 더해지면 의미 있는 성능 향상이 가능함을 확인하였다.